Bases de Datos 2

Teoría 02

Julián Grigera

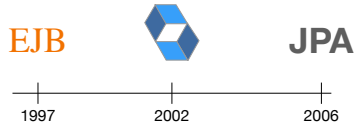
Hibernate y JPA

Java Persistence API

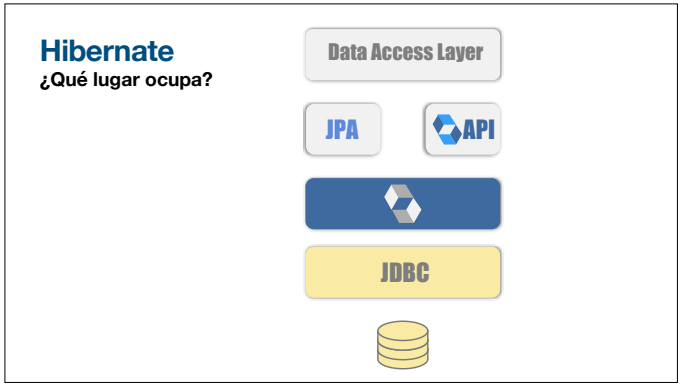
JPA es un estándar Basado en Hibernate, EJB y otros productos.
Hibernate es la implementación de referencia, si bien existen otras.

Java Persistence API

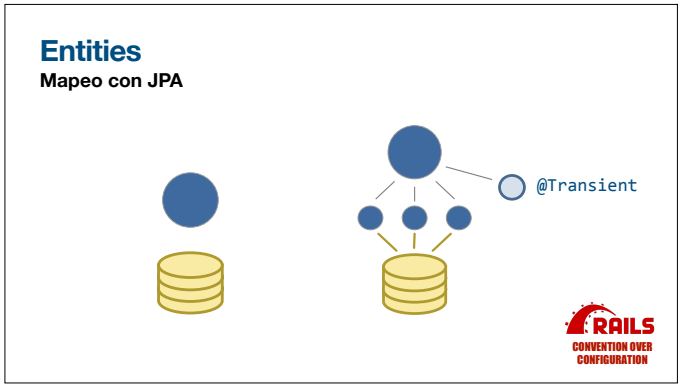
JPA



JPA toma ideas de EJB / Hibernate / etc.
Por este motivo, tiene pocas especificaciones.
Hibernate es la implementación de referencia.

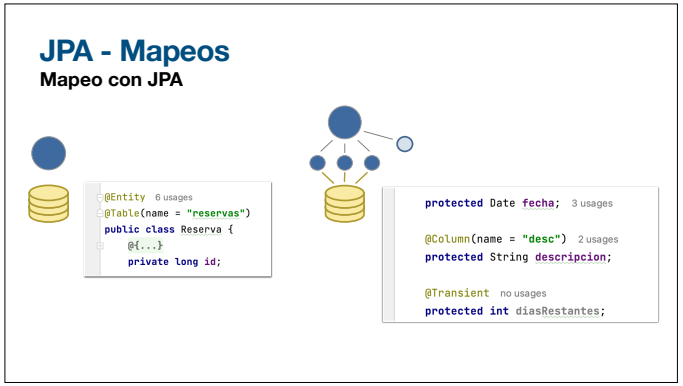


Hibernate: ORM popular. Es un estándar de facto de la industria.
JPA es posterior, y está creado con una alta influencia de Hibernate, dado su predominio.
Si se utiliza JPA, hay suficiente tecnología para lograr independencia (es decir, evitar queries explícitas, *saves*, etc)



Entidades en JPA: POJOs persistentes.
Por default en JPA se persisten todos los atributos, a menos que se marquen como @Transient - un ejemplo de Convention over Configuration

---



Ejemplos de código

- Las entidades tienen opciones para configurar, como el nombre de la tabla (útil para evitar palabras reservadas del DBMS)
- Los atributos también, pueden especificarse propiedades de la columna, restricciones, etc
- También se puede ligar @Column a un getter/método

Hibernate y JPA
Mapping

JPA es un estándar Basado en Hibernate, EJB y otros productos.
Hibernate es la implementación de referencia, si bien existen otras.

Jerarquías
Mapeo con JPA

SINGLE_TABLE

```
@Entity(name = "clientes")  7 usages
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo")
public abstract class  Cliente {
    // ...
}
```

```
@Entity  3 usages
@DiscriminatorValue("1")
public class ClienteNormal extends Cliente{

    // ...
}
```

```
@Entity  5 usages
@DiscriminatorValue("2")
public class ClienteVIP extends Cliente {

    // ...
}
```

Jerarquías
Mapeo con JPA

JOINED

```
@Entity(name = "clientes")  7 usages
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class  Cliente {

    // ...
}
```

```
@Entity  3 usages
public class ClienteNormal extends Cliente{

    // ...
}
```

```
@Entity  3 usages
public class ClienteVIP extends Cliente{

    // ...
}
```

Jerarquías
Mapeo con JPA

```
@Entity  8 usages
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class  Cliente {
```

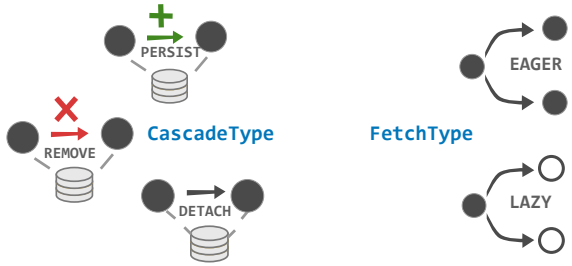
TABLE_PER_CLASS

```
@Entity  3 usages
public class ClienteNormal extends Cliente{

    // ...
}
```

```
@Entity  3 usages
public class ClienteVIP extends Cliente{

    // ...
}
```

En TABLE_PER_CLASS la superclase se marca como Entity pero no se indica nada relacionado a la tabla - dado que no se generará

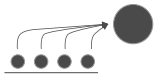
Relaciones
Mapeo con JPA



- **Cascade** establece la persistencia por alcance. Por defecto nada se propaga en JPA. Puede ser ALL, PERSIST, MERGE, REMOVE, REFRESH, DETACH
- El tipo de **Fetch** puede ser **EAGER** o **LAZY**, cada uno puede ser más o menos performante según el caso.
- El valor de tanto Cascade como Fetch dependerá del uso.

Relaciones
Many to Many - to One

```
@Entity
@Table(name = "BOOKS")
public class Book {

    @ManyToMany
    private Set<Author> authors;

    ...
}
```



```
@Entity
public class Book {

    @ManyToOne(fetch = FetchType.LAZY)
    private Publisher publisher;

    ...
}
```

ManyToMany: No requiere especificación de tipo (como Hibernate)
ManyToOne

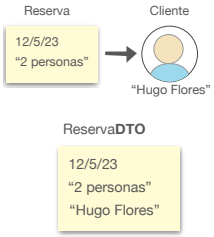
Relaciones
One to Many

```
@Entity(name = "Post")
@Table(name = "post")
public class Post {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @OneToMany(
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<PostComment> comments = new ArrayList<>();
```

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private List<PostComment> comments = new ArrayList<>();
```

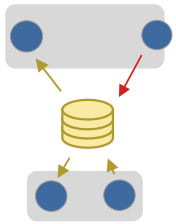
Mapeo por default (tablaintermedia)
Join Column

DTOs
JPA



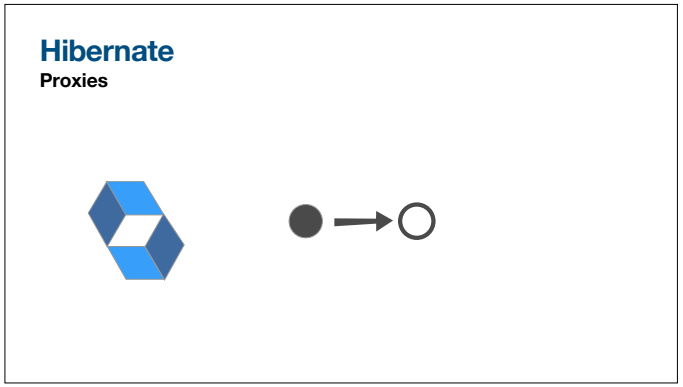
- No puedo leer objetos fuera de una tx, qué hago?
- DTOs. Según Fowler: “An object that carries data between processes in order to reduce the number of method calls.”
- Se puede aprovechar el hecho de crear otra clase y cambiar el formato, agregar atributos externos, etc.

Concurrencia
JPA



¿Cómo se manejan los problemas de concurrencia?
Se observa el @Version al escribir,
- si el nro es el mismo de la base, ok
- si no, tengo una versión vieja << hasta ahí llega JPA - solo eleva excepción

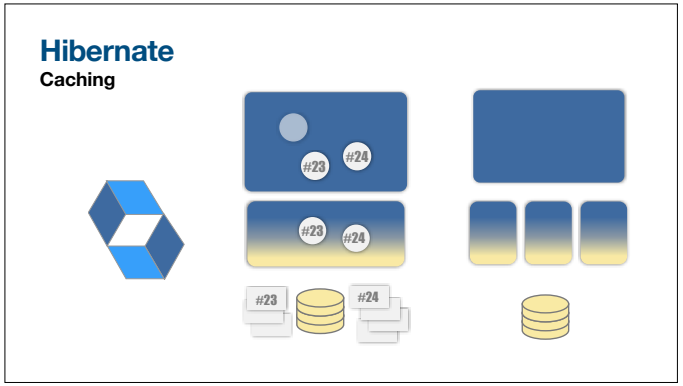
JPA es un estándar Basado en Hibernate, EJB y otros productos.
Hibernate es la implementación de referencia, si bien existen otras.



Qué pasa si…

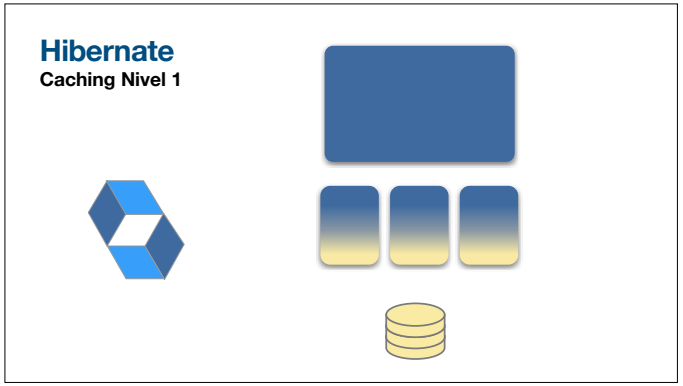
1. Abro una sesión, una tx y guardo un objeto “reserva” relacionado a otro, “cliente” de forma LAZY
2. Abro otra tx y recupera “reserva”
3. Pido el “cliente” a la reserva recuperada
4. Cierro sesión

El cliente, debería ser un proxy? O un objeto del modelo?
Y si hago los pasos 2-3 en una nueva sesión?



Tiene soporte para:

- Caching
    - Nivel 1 - activado por default, depende de cada sesión
    - Nivel 2 - solamente un hotspot, con implementaciones para descargar
        - Incluye caching de queries



Nivel 1

Cuando una entidad se carga o actualiza por primera vez en una sesión, se almacena en la caché a nivel de sesión.

Solicitudes posteriores de la misma entidad en la misma sesión se sirven desde la caché

Se minimizan así accesos a la BD

Nivel 2

A nivel de SessionFactory - si no se halla un objeto en la **CacheL1**, se busca en **CacheL2**

No viene implementada, pero hay productos que lo hacen

Se pueden cachear resultados de consultas HQL